INSTITUTE OF AERONAUTICAL ENGINEERING
AN ASSIGNMENT

REPORT OF Neural Computing and Deep Learning

COURSE CODE- ACSD32

BY

D. GUNA

23951A052T

COMPUTER SCIENCE AND ENGINEERING

INSTITUTE OF AERONAUTICAL ENGINEERING COLLEGE,
DUNDIGAL, HYDERABAD-500043, TELANGANA, INDIA

Counter Propagation Network Kohonen + Grossberg Layers

1. Introduction

A Multi-Layer Perceptron (MLP) is a type of artificial neural network that is widely used for supervised learning tasks such as:

- Classification (e.g., digit recognition)
- Regression (e.g., predicting prices)
- Pattern recognition (e.g., image or speech recognition)

An MLP consists of:

1. Input Layer: Receives the raw input features.
2. Hidden Layer(s): One or more layers that process inputs using neurons with activation functions.
3. Output Layer: Produces the final prediction.

Forward computation is the process of passing input through the network to compute the output.

---

2. Structure of an MLP

Components:

1. Neurons (nodes): Compute weighted sums of inputs plus a bias.
2. Weights (W): Parameters that define the strength of connections between neurons.
3. Bias (b): Constant term added to weighted sum to allow shifting the activation function.
4. Activation Function (f): Non-linear function applied to neuron output to capture complex patterns.

Mathematical Representation of a Neuron:

zj=∑i=1nwijxi+bjz_j = \sum_{i=1}^{n} w_{ij} x_i + b_jzj=i=1∑nwijxi+bj
aj=f(zj)a_j = f(z_j)aj=f(zj)

Where:

- xix_ixi = input to the neuron
- wijw_{ij}wij = weight connecting input iii to neuron jjj
- bjb_jbj = bias for neuron jjj
- zjz_jzj = weighted sum
- aja_jaj = output after activation

3. Forward Computation in MLP

Step 1: Compute Hidden Layer Output

1. For each neuron in the hidden layer:
    - Compute weighted sum zj=∑iwijxi+bjz_j = \sum_i w_{ij} x_i + b_jzj=∑iwijxi+bj
    - Apply activation function aj=f(zj)a_j = f(z_j)aj=f(zj)

Activation Functions Commonly Used:

Function	Formula	Output Range
Sigmoid	f(z)=11+e−zf(z) = \frac{1}{1 + e^{-z}}f(z)=1+e−z1	(0,1)
ReLU	f(z)=max⁡(0,z)f(z) = \max(0,z)f(z)=max(0,z)	[0, ∞)
Tanh	f(z)=tanh⁡(z)f(z) = \tanh(z)f(z)=tanh(z)	(-1,1)

Example Hidden Layer Calculation:

- Inputs: x=[1,2]x = [1, 2]x=[1,2]
- Weights: W=[0.5−0.40.30.2]W = \begin{bmatrix}0.5 &-0.4 \\ 0.3 & 0.2\end{bmatrix}W=[0.50.3−0.40.2]
- Biases: b=[0.1,−0.2]b = [0.1, -0.2]b=[0.1,−0.2]
- Activation: Sigmoid

Compute z1=1∗0.5+2∗0.3+0.1=1.2z_1 = 1*0.5 + 2*0.3 + 0.1 = 1.2z1 =1∗0.5+2∗0.3+0.1=1.2
a1=11+e−1.2≈0.768a_1 = \frac{1}{1 + e^{-1.2}} \approx 0.768a1=1+e−1.21 ≈0.768

Compute z2=1∗(−0.4)+2∗0.2+(−0.2)=−0.2z_2 = 1*(-0.4) + 2*0.2 + (-0.2) = -0.2z2 =1∗(−0.4)+2∗0.2+(−0.2)=−0.2
a2=11+e0.2≈0.450a_2 = \frac{1}{1 + e^{0.2}} \approx 0.450a2=1+e0.21≈0.450

Hidden layer output: [0.768,0.450][0.768, 0.450][0.768,0.450]

Step 2: Compute Output Layer

1. For each neuron in the output layer:
    - Compute weighted sum: zk=∑jwjkaj+bkz_k = \sum_j w_{jk} a_j + b_kzk=∑jwjkaj+bk
    - Apply activation function (e.g., softmax for multi-class classification, linear for regression)

Example Output Layer:

- Hidden outputs: a=[0.768,0.450]a = [0.768, 0.450]a=[0.768,0.450]
- Output weights: Wout=[0.7,−0.6]W_{out} = [0.7, -0.6]Wout=[0.7,−0.6]
- Output bias: bout=0.05b_{out} = 0.05bout=0.05

Compute
zout=0.768∗0.7+0.450∗(−0.6)+0.05=0.5376−0.27+0.05=0.3176z_{out} = 0.768*0.7 + 0.450*(-0.6) + 0.05 = 0.5376 - 0.27 + 0.05 = 0.3176zout =0.768∗0.7+0.450∗(−0.6)+0.05=0.5376−0.27+0.05=0.3176

Apply sigmoid: y=11+e−0.3176≈0.579y = \frac{1}{1 + e^{-0.3176}} \approx 0.579y=1+e−0.31761≈0.579

Final network output = 0.579

4. General MLP Forward Propagation Equations

For an MLP with L layers, we define:

z(l)=W(l)a(l−1)+b(l)z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}z(l)=W(l)a(l−1)+b(l)
a(l)=f(z(l))a^{(l)} = f(z^{(l)})a(l)=f(z(l))

Where:

- l=1,2,...,Ll = 1,2,...,Ll=1,2,...,L

- a(0)=xa^{(0)} = xa(0)=x (input vector)
- W(l)W^{(l)}W(l) = weights of layer l
- b(l)b^{(l)}b(l) = biases of layer l
- fff = activation function

Forward computation consists of iteratively applying these equations from input to output layer.

5. Example: Multi-Layer Forward Pass

Network:

- Input layer: 2 neurons
- Hidden layer 1: 2 neurons, ReLU activation
- Hidden layer 2: 2 neurons, Sigmoid activation
- Output layer: 1 neuron, Sigmoid activation

Inputs:

x=[1,2]x = [1, 2]x=[1,2]

Weights & Biases:

- Hidden 1: W1=[0.5−0.40.30.2],b1=[0.1,−0.2]W_1 = \begin{bmatrix}0.5 & -0.4\\ 0.3 & 0.2\end{bmatrix}, b_1=[0.1,-0.2]W1=[0.50.3−0.40.2],b1 =[0.1,−0.2]
- Hidden 2: W2=[0.6−0.10.20.5],b2=[0.05,0.05]W_2 = \begin{bmatrix}0.6 & -0.1 \\ 0.2 & 0.5\end{bmatrix}, b_2=[0.05,0.05]W2=[0.60.2−0.10.5],b2=[0.05,0.05]
- Output: W3=[0.7,−0.3],b3=0.1W_3 = [0.7, -0.3], b_3 = 0.1W3 =[0.7,−0.3],b3=0.1

Step 1: Hidden Layer 1

z(1)=W1x+b1=[0.5∗1+−0.4∗2+0.10.3∗1+0.2∗2−0.2]=[−0.20.5]z^{(1)} = W_1 x + b_1 = \begin{bmatrix}0.5*1 + -0.4*2 + 0.1\\ 0.3*1 + 0.2*2 - 0.2\end{bmatrix} = \begin{bmatrix}-0.2\\ 0.5\end{bmatrix}z(1)=W1x+b1 =[0.5∗1+−0.4∗2+0.10.3∗1+0.2∗2−0.2]=[−0.20.5]

ReLU: a(1)=[0,0.5]a^{(1)} = [0, 0.5]a(1)=[0,0.5]

Step 2: Hidden Layer 2

z(2)=W2a(1)+b2=[0.6∗0+−0.1∗0.5+0.050.2∗0+0.5∗0.5+0.05]=[−0.05+0.050.25+0.05]=[0,0.3]z^{(2)} = W_2 a^{(1)} + b_2 = \begin{bmatrix}0.6*0 + -0.1*0.5 + 0.05\\ 0.2*0 + 0.5*0.5 + 0.05\end{bmatrix} = \begin{bmatrix}-0.05+0.05 \\ 0.25+0.05\end{bmatrix} = [0,0.3]z(2)=W2a(1)+b2 =[0.6∗0+−0.1∗0.5+0.050.2∗0+0.5∗0.5+0.05]=[−0.05+0.050.25+0.05]=[0,0.3]

Sigmoid: a(2)=[0.5,0.574]a^{(2)} = [0.5, 0.574]a(2)=[0.5,0.574]

Step 3: Output Layer

z(3)=W3a(2)+b3=0.7∗0.5+(−0.3∗0.574)+0.1=0.35−0.1722+0.1=0.2778z^{(3)} = W_3 a^{(2)} + b_3 = 0.7*0.5 + (-0.3*0.574) + 0.1 = 0.35 -0.1722 +0.1 = 0.2778z(3)=W3a(2)+b3=0.7∗0.5+(−0.3∗0.574)+0.1=0.35−0.1722+0.1=0.2778

Sigmoid: y=0.569y = 0.569y=0.569

Final output: 0.569

---

6. Summary of Forward Computation Steps

1. Start with input vector xxx
2. For each layer lll:
    - Compute weighted sum z(l)=W(l)a(l−1)+b(l)z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}z(l)=W(l)a(l−1)+b(l)
    - Apply activation a(l)=f(z(l))a^{(l)} = f(z^{(l)})a(l)=f(z(l))
3. Output layer produces prediction y=a(L)y = a^{(L)}y=a(L)

Forward pass is crucial for:

- Computing network output
- Backpropagation during training

7. Importance and Applications

- Training Neural Networks: Forward pass is the first step before computing loss and gradients.

- **Prediction: Forward computation is used to infer outputs from trained networks.**
- **Understanding Network Behavior: Helps in debugging, visualization, and analyzing hidden layers.**
- **Real-world Applications:**
    - **Handwriting recognition**
    - **Speech recognition**
    - **Stock market prediction**
    - **Medical diagnosis**